

OPTION B INSTITUTE FAMILY

M08 - FILES, APIS & DATA INGESTION

Standalone Zero-to-Job-Ready Data Engineer 2026

BEGINNER-FIRST MASTER REPAIR RC4

17 canonical lessons | full 16-stage teacher contract | protected attempt-first
review

This learner book is rebuilt from the frozen canonical technical material. It is not a front-matter patch: every lesson is re-laid out into the shared institute teaching grammar while preserving the course competency, source route, guided work, independent proof and validation boundary.

This book is your teacher

Use the book in order. External videos and official docs are visual companions, not hidden prerequisites. Watch or read only the assigned scope, then close it before the independent task.

The 16-stage lesson loop

Why it matters -> What you learn -> Prerequisite -> Picture it -> New words -> Useful source -> Follow once -> Expected result -> Why it worked -> Beginner mistakes -> Recovery -> Independent task -> Validate -> Teach it back -> Protected review + changed retry -> Evidence/mastery + exact next route.

Attempt-first protection

Never open a protected lesson review before saving a genuine independent attempt. If repair is needed, compare only the weak idea, close the review, solve a changed retry, then validate again. A solution seen during review is repair evidence, not mastery.

What counts as mastery

A reproducible artifact or explanation, an independent validation control, preserved failure/recovery evidence, a changed transfer task completed without the worked answer open, and an own-words explanation. Percent-complete is not a substitute.

How to use your devices

Phone: assigned video/reference, reading, revision. Laptop/PC: every real command, query, file, workbook, code task, validation and evidence artifact. Install only what the current lesson asks for.

Contents / Index

This book is your teacher	2
Contents / Index	3
Prerequisite / tool map	4
Practice / review / Gate route	4
DE08-01 - Ingestion architecture: source -> landing -> staging	5
DE08-02 - Defensive CSV ingestion	7
DE08-03 - JSON and nested-record ingestion	9
DE08-04 - Parquet ingestion and metadata inspection	11
DE08-05 - REST API contracts, status codes and timeouts	13
DE08-06 - API authentication, pagination and rate limits	15
DE08-07 - Retries, backoff, jitter and transient failures	17
DE08-08 - Database extraction and safe query boundaries	19
DE08-09 - Historical batch landing and raw metadata	21
DE08-10 - Incremental loads and high-water marks	23
DE08-11 - CDC concepts and change-event semantics	25
DE08-12 - Schema drift, compatibility, time zones and quarantine	27
DE08-13 - Checkpoint/state management and replay safety	29
DE08-14 - Idempotent landing and content identity	31
DE08-15 - Late-arriving data, event time and processing time	33
DE08-16 - Ingestion observability: counts, latency and run metadata	35
DE08-17 - Ingestion integration, reconciliation and backfill proof	37
Module mastery checklist + revision quick reference	39

Prerequisite / tool map

Ingest files, APIs and databases with reliable state, retries, quarantine and replay safety.

Item	Rule
Entry prerequisite	Previous module/Gate evidence must be stable.
Primary device	PC/laptop for hands-on work; phone is support only.
External sources	Optional/supporting unless the lesson explicitly marks an official current reference.
Evidence	Module practical + fresh retest + saved evidence; no critical unresolved failure.

Practice / review / Gate route

Lesson teaching -> follow once -> independent attempt -> validate -> save evidence. Only after the genuine attempt may protected review open. If weak: targeted repair -> close review -> changed retry -> revalidate. Finish the module practical/retest route before any downstream Gate that depends on this module.

DE08-01 - Ingestion architecture: source -> landing -> staging

1. Why it matters

This matters because later Files, APIs & Data Ingestion work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Design a reproducible source-to-landing-to-staging boundary and state what is immutable, validated and transformed at each layer. Teach it from zero Ingestion is a chain of responsibility. Source is what the producer exposes. Landing preserves raw evidence as received. Staging parses and validates it into a shape safe for later transformation. Do not overwrite raw bytes just because a later schema is easier to query. A trustworthy pipeline can show what arrived, what was rejected, and what changed afterward.

3. Prerequisite / starting state

M07 frozen; M04-M06 Python/file/Git basics available.

4. Picture it in your head

VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt Whole workshop architecture pass: APIs/files/databases -> normalization -> warehouse; ignore platform-specific install steps. Starting state / prerequisite M07 frozen; M04-M06 Python/file/Git basics available.

5. New words before we use them

VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt Whole workshop architecture pass: APIs/files/databases -> normalization -> warehouse; ignore platform-specific install steps. Starting state / prerequisite M07 frozen; M04-M06 Python/file/Git basics available. Outcome Design a reproducible source-to-landing-to-staging boundary and state what is immutable, validated and transformed at each layer.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use matching data-ingestion / script / database chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Create folders source/, landing/, staging/, quarantine/. Copy one batch file into landing without renaming its bytes; record size + SHA-256; parse it into staging only after validation. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

11. If you get stuck

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

12. Small independent task

Design a 4-layer folder/contract for a weather-station file feed. State exactly what can change at each boundary and what evidence must never be overwritten.

13. Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

14. Teach it back

Explain ingestion architecture: source -> landing -> staging in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success. Current authoritative reference(s) dlt REST API tutorial Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

M08 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-02

DE08-02 - Defensive CSV ingestion

1. Why it matters

This matters because later Files, APIs & Data Ingestion work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Read CSV defensively with explicit encoding/header/delimiter/schema expectations, quarantine malformed business rows and reconcile counts. Teach it from zero CSV is deceptively simple: delimiters, quotes, newlines, encodings, headers, missing fields and business-invalid values can all break assumptions. Defensive ingestion separates parsing validity from business validity, counts every source business row, and quarantines bad rows with a reason instead of silently dropping them.

3. Prerequisite / starting state

DE08-01 mastered.

4. Picture it in your head

VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt File-ingestion portions only; course lab owns malformed-row/quarantine rules. Starting state / prerequisite DE08-01 mastered.

5. New words before we use them

VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt File-ingestion portions only; course lab owns malformed-row/quarantine rules. Starting state / prerequisite DE08-01 mastered. Outcome Read CSV defensively with explicit encoding/header/delimiter/schema expectations, quarantine malformed business rows and reconcile counts.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use matching data-ingestion / script / database chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Use `csv.DictReader`. Require the expected header, parse `ship_date` as a real date and `weight_kg` as numeric. Send bad rows to quarantine and prove seen = accepted + quarantined. Open the matching Guided Labs section. Reproduce the pattern once with the normal `ParceIFlow` fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

11. If you get stuck

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

12. Small independent task

Write a CSV reader for a changed shipment file with one invalid date and one missing required ID. Quarantine errors with reason and prove count reconciliation.

13. Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

14. Teach it back

Explain defensive csv ingestion in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success. Current authoritative reference(s) Python 3.14 csv pandas read_csv Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

M08 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-03

DE08-03 - JSON and nested-record ingestion

1. Why it matters

This matters because later Files, APIs & Data Ingestion work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Parse nested JSON without silently dropping arrays/objects, flatten only the fields required by the target grain and preserve raw payload evidence. Teach it from zero JSON can represent nested objects and arrays, so flattening changes shape. Decide the target grain first. Preserve the raw payload, then extract only needed paths. An event array normally means one output row per event, not one row per whole document. Missing nested keys need an explicit policy.

3. Prerequisite / starting state

DE08-02 mastered.

4. Picture it in your head

VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt API/nested-data normalization portions; course lab owns exact flattening grain. Starting state / prerequisite DE08-02 mastered.

5. New words before we use them

VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt API/nested-data normalization portions; course lab owns exact flattening grain. Starting state / prerequisite DE08-02 mastered. Outcome Parse nested JSON without silently dropping arrays/objects, flatten only the fields required by the target grain and preserve raw payload evidence.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use matching data-ingestion / script / database chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Flatten normal/input/tracking_events.json to one row per tracking event with event_id, shipment_id, region, scan_type, event_time and device_id. Keep the original JSON untouched. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

11. If you get stuck

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

12. Small independent task

Flatten a changed nested JSON payload whose root contains devices[] and each device contains readings[]. State output grain before code.

13. Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

14. Teach it back

Explain json and nested-record ingestion in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success. Current authoritative reference(s) Python 3.14 json pandas read_json Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

M08 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-04

DE08-04 - Parquet ingestion and metadata inspection

1. Why it matters

This matters because later Files, APIs & Data Ingestion work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Explain columnar Parquet layout, verify file identity/magic/footer metadata, and inspect schema/row-group metadata with a compatible reader when available. Teach it from zero Parquet is a column-oriented binary format. A file contains column chunks organized in row groups plus footer metadata; readers use metadata to locate needed columns. The format begins and ends with PAR1 magic. Metadata inspection is not the same as opening a text file. A compatible Parquet engine such as PyArrow exposes schema and row-group details.

3. Prerequisite / starting state

DE08-03 mastered.

4. Picture it in your head

VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt File/warehouse portions as visual context only; Apache Parquet docs are format authority. Starting state / prerequisite DE08-03 mastered.

5. New words before we use them

VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt File/warehouse portions as visual context only; Apache Parquet docs are format authority. Starting state / prerequisite DE08-03 mastered. Outcome Explain columnar Parquet layout, verify file identity/magic/footer metadata, and inspect schema/row-group metadata with a compatible reader when available.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp 2026 — From APIs to Warehouses with dlt

Assigned scope: Long-form main class — file/API ingestion and warehouse loading; use Parquet/file portions when reached

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

Note: Long-form class is allowed. Watch coherently, then use the lesson-specific book/lab and do the independent task without the video.

[Open source](#)

7. Follow with me once

Run `m08_parquet_probe.py` on the supplied real Apache Parquet fixture. Prove magic at both ends and footer metadata length. If PyArrow is installed, also record row count, row groups and schema; otherwise record the dependency honestly. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

11. If you get stuck

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

12. Small independent task

Inspect the supplied Parquet file. Without a reader, prove binary identity/footer safely; with PyArrow, record schema/row groups/rows. Explain what you could not verify if PyArrow is unavailable.

13. Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

14. Teach it back

Explain parquet ingestion and metadata inspection in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success. Current authoritative reference(s) Apache Parquet overview Apache Parquet metadata pandas read_parquet Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

M08 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-05

DE08-05 - REST API contracts, status codes and timeouts

1. Why it matters

This matters because later Files, APIs & Data Ingestion work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Call an HTTP API with explicit timeout/status handling and prove that JSON decoding alone is not success evidence. Teach it from zero HTTP success is a contract, not "I got JSON". A server can return JSON error details with a 4xx/5xx status. Set a timeout, check status before trusting payload, separate connection/timeouts from HTTP errors, and keep enough request metadata to reproduce a failure without logging secrets.

3. Prerequisite / starting state

DE08-04 mastered.

4. Picture it in your head

VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt API extraction portions; Requests docs + localhost lab define timeout/status contract. Starting state / prerequisite DE08-04 mastered.

5. New words before we use them

VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt API extraction portions; Requests docs + localhost lab define timeout/status contract. Starting state / prerequisite DE08-04 mastered. Outcome Call an HTTP API with explicit timeout/status handling and prove that JSON decoding alone is not success evidence.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use matching data-ingestion / script / database chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Call the localhost /health and /records endpoints. Prove /records without auth is 401. Call /slow with a short timeout and save the timeout evidence. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

11. If you get stuck

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

12. Small independent task

Write a function that accepts URL + timeout, rejects non-2xx responses before trusting JSON, and distinguishes HTTP error from timeout/connection error.

13. Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

14. Teach it back

Explain rest api contracts, status codes and timeouts in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success. Current authoritative reference(s) Requests Quickstart - status/timeouts Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

M08 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-06

DE08-06 - API authentication, pagination and rate limits

1. Why it matters

This matters because later Files, APIs & Data Ingestion work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Keep credentials out of code, traverse pagination exactly once, obey rate-limit signals and reconcile fetched record counts. Teach it from zero Credentials belong in environment/configuration boundaries, not committed source. Pagination means one logical dataset spans several responses. Follow the API next-page contract until it ends, protect against loops, and reconcile fetched IDs/counts. Rate limits are part of the source contract; respect Retry-After or documented quota behavior.

3. Prerequisite / starting state

DE08-05 mastered.

4. Picture in your head

VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt API extraction/pagination concepts; course localhost API proves token/pagination/rate-limit behavior. Starting state / prerequisite DE08-05 mastered.

5. New words before we use them

VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt API extraction/pagination concepts; course localhost API proves token/pagination/rate-limit behavior. Starting state / prerequisite DE08-05 mastered. Outcome Keep credentials out of code, traverse pagination exactly once, obey rate-limit signals and reconcile fetched record counts.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp 2026 — From APIs to Warehouses with dlt
Assigned scope: Long-form main class — API extraction, authentication/pagination/rate-limit reliability concepts

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

Note: The workshop is a main class, not a note-taking assignment. Type/run; do not transcribe.

[Open source](#)

7. Follow with me once

Fetch all localhost records with Bearer token m08-demo-token. The data spans three logical pages. Save IDs and prove no duplicate page/record traversal. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

11. If you get stuck

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

12. Small independent task

Build a paginator that stops only when `next_page` is null, refuses a repeated page token, keeps token outside source code and reconciles unique IDs.

13. Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

14. Teach it back

Explain api authentication, pagination and rate limits in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success. Current authoritative reference(s) Requests authentication dlt REST API source pagination Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

M08 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-07

DE08-07 - Retries, backoff, jitter and transient failures

1. Why it matters

This matters because later Files, APIs & Data Ingestion work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Retry only appropriate transient/idempotent operations with bounded attempts, exponential backoff/jitter and observable retry evidence. Teach it from zero Retries help only when failure may be transient. Bound attempts. Retry appropriate idempotent requests/statuses, not every error. Exponential backoff reduces repeated pressure; jitter prevents many clients from retrying in lockstep. Every retry should be observable so hidden instability is not mistaken for a clean run.

3. Prerequisite / starting state

DE08-06 mastered.

4. Picture it in your head

VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt Reliability/failure-handling concepts; urllib3 docs + lab define bounded retry behavior. Starting state / prerequisite DE08-06 mastered.

5. New words before we use them

VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt Reliability/failure-handling concepts; urllib3 docs + lab define bounded retry behavior. Starting state / prerequisite DE08-06 mastered. Outcome Retry only appropriate transient/idempotent operations with bounded attempts, exponential backoff/jitter and observable retry evidence.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp 2026 — From APIs to Warehouses with dlt
Assigned scope: Long-form main class — reliable ingestion/failure handling; course lab adds explicit backoff/jitter evidence
How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.
Note: Workshop provides the visual workflow; our task proves retry behavior.
[Open source](#)

7. Follow with me once

Use the guided client against the localhost server. Page 2 rate-limits once and page 3 returns one temporary 503. Prove the client eventually retrieves all records and logs exactly which attempts were retried. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcellFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

11. If you get stuck

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

12. Small independent task

Implement bounded retry for GET on 429/503 only. Record attempt number/status/backoff. Explain why blindly retrying POST can be unsafe.

13. Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

14. Teach it back

Explain retries, backoff, jitter and transient failures in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success. Current authoritative reference(s) urllib3 Retry Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

M08 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-08

DE08-08 - Database extraction and safe query boundaries

1. Why it matters

This matters because later Files, APIs & Data Ingestion work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Extract from a database with parameterized read queries, explicit columns, deterministic bounds and no accidental source mutation. Teach it from zero Database extraction should be boring and safe: explicit columns, read-only SELECT logic, parameters instead of string-built values, deterministic ordering/bounds, and a known extraction grain. Large production extracts often require chunking/snapshots/replica strategies, but the learner must first prove boundary correctness.

3. Prerequisite / starting state

DE08-07 mastered; M07 database modeling complete.

4. Picture it in your head

VIDEO FIRST route DataTalksClub - Docker/PostgreSQL data-engineering workshop Use only Python/database ingestion/query-boundary portions. DO NOT follow Docker setup in M08; Docker is M10.

5. New words before we use them

VIDEO FIRST route DataTalksClub - Docker/PostgreSQL data-engineering workshop Use only Python/database ingestion/query-boundary portions. DO NOT follow Docker setup in M08; Docker is M10. Starting state / prerequisite DE08-07 mastered; M07 database modeling complete.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use matching data-ingestion / script / database chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Query source_extract.db for orders updated from 10:00 inclusive to 12:30 exclusive using parameter placeholders. Save row count and SQL text. Do not UPDATE/DELETE the source. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

11. If you get stuck

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

12. Small independent task

Parameterize a time-window SELECT against the SQLite source. Return explicit columns and deterministic order; save query + parameters + count.

13. Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

14. Teach it back

Explain database extraction and safe query boundaries in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success. Current authoritative reference(s) Python 3.14 sqlite3 Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

M08 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-09

DE08-09 - Historical batch landing and raw metadata

1. Why it matters

This matters because later Files, APIs & Data Ingestion work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Land historical files without overwriting raw bytes; capture source name, size, hash, ingest time and batch/business date metadata. Teach it from zero Historical backfills are easiest to trust when landing preserves each source object exactly once and metadata explains identity. Useful metadata includes original name/path, size, content hash, ingest time, business/batch date and source system. Landing is evidence, not a convenient scratch folder.

3. Prerequisite / starting state

DE08-08 mastered.

4. Picture it in your head

VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt Source/file ingestion and durable loading mental model; course lab defines immutable landing metadata. Starting state / prerequisite DE08-08 mastered.

5. New words before we use them

VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt Source/file ingestion and durable loading mental model; course lab defines immutable landing metadata. Starting state / prerequisite DE08-08 mastered. Outcome Land historical files without overwriting raw bytes; capture source name, size, hash, ingest time and batch/business date metadata.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use matching data-ingestion / script / database chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Land the three batch files to a clean landing directory. Record SHA-256 and business-row count for each. Rerun landing and prove existing bytes are unchanged. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParceleFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

11. If you get stuck

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

12. Small independent task

Land three changed batch files under a run/date prefix. Save name, bytes, SHA-256, business rows and ingest timestamp; rerun without overwriting changed bytes.

13. Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

14. Teach it back

Explain historical batch landing and raw metadata in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success. Current authoritative reference(s) Python 3.14 pathlib Python 3.14 hashlib Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

M08 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-10

DE08-10 - Incremental loads and high-water marks

1. Why it matters

This matters because later Files, APIs & Data Ingestion work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Load only new/changed rows using a stable cursor/high-water rule that does not lose ties at the boundary. Teach it from zero A high-water mark tells the next run where to continue. Timestamp-only cursors can lose rows when several records share the same timestamp. A composite cursor such as (updated_at, record_id) gives a deterministic total order. State should move only after the destination commit succeeds.

3. Prerequisite / starting state

DE08-09 mastered.

4. Picture it in your head

VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt Incremental/state concepts; current dlt incremental docs define cursor/state model. Starting state / prerequisite DE08-09 mastered.

5. New words before we use them

VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt Incremental/state concepts; current dlt incremental docs define cursor/state model. Starting state / prerequisite DE08-09 mastered. Outcome Load only new/changed rows using a stable cursor/high-water rule that does not lose ties at the boundary.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use matching data-ingestion / script / database chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Process incremental_source.csv in two runs: first five rows, then the remainder. Use (updated_at, record_id) as cursor. Prove I006 is not lost even though I004-I006 share 10:10. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

11. If you get stuck

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

12. Small independent task

Process the 8-row incremental fixture using composite cursor. Stop first run after five rows, then finish; prove tie-boundary row I006 is not lost and third run returns zero.

13. Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

14. Teach it back

Explain incremental loads and high-water marks in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success. Current authoritative reference(s) dlt incremental loading dlt cursor incremental Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

M08 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-11

DE08-11 - CDC concepts and change-event semantics

1. Why it matters

This matters because later Files, APIs & Data Ingestion work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Interpret insert/update/delete change events, before/after images and source ordering metadata without pretending a batch poll is CDC. Teach it from zero CDC captures changes from a transaction/change log and emits operation semantics such as create/update/delete, often with before/after state and source ordering metadata. Polling a table by `updated_at` is incremental extraction, not true CDC. Downstream consumers must define delete handling, ordering and replay behavior.

3. Prerequisite / starting state

DE08-10 mastered.

4. Picture it in your head

VIDEO FIRST route Binod Suman Academy - Change Data Capture with Debezium/Kafka What/why/how CDC + Debezium/Kafka demo; course book controls generic event semantics. Starting state / prerequisite DE08-10 mastered.

5. New words before we use them

VIDEO FIRST route Binod Suman Academy - Change Data Capture with Debezium/Kafka What/why/how CDC + Debezium/Kafka demo; course book controls generic event semantics. Starting state / prerequisite DE08-10 mastered. Outcome Interpret insert/update/delete change events, before/after images and source ordering metadata without pretending a batch poll is CDC.

6. Useful video / source

Source: Binod Suman Academy — Change Data Capture with Debezium/Kafka

Assigned scope: What/why/how CDC + Debezium/MySQL/Kafka demo

How to use it: Use for the CDC event mental model and real demo. Teaching Book remains authoritative for generic change-event semantics and failure reasoning.

Note: Published 2025; tool-specific demo used to make the abstract CDC concept visible.

[Open source](#)

7. Follow with me once

Apply `cdc_events.jsonl` in seq order to an in-memory current-state map. Prove A1 is updated, A2 is deleted and A3 exists. Explain why the event log remains valuable even after current state is reconstructed. Open the matching Guided Labs section. Reproduce the pattern once with the normal `ParcelFlow` fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

11. If you get stuck

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

12. Small independent task

Consume the supplied CDC log to derive current state plus an audit trail. State delete policy and what sequence field protects.

13. Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

14. Teach it back

Explain cdc concepts and change-event semantics in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success. Current authoritative reference(s) Debezium architecture Debezium event flattening Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

M08 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-12

DE08-12 - Schema drift, compatibility, time zones and quarantine

1. Why it matters

This matters because later Files, APIs & Data Ingestion work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Detect compatible/incompatible drift, normalize timestamps to UTC, quarantine invalid rows and avoid silently coercing bad values. Teach it from zero Schema drift is any change to expected fields/types/meaning. Some changes are additive/compatible; others break consumers. Timestamp strings also carry semantics: normalize instants to UTC while preserving enough raw evidence to diagnose source issues. Invalid rows go to quarantine with reason; do not replace bad numeric/time values with zero.

3. Prerequisite / starting state

DE08-11 mastered.

4. Picture it in your head

quarantine VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt Schema normalization/change validation concepts; course lab adds UTC/quarantine contracts. Starting state / prerequisite DE08-11 mastered.

5. New words before we use them

quarantine VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt Schema normalization/change validation concepts; course lab adds UTC/quarantine contracts. Starting state / prerequisite DE08-11 mastered. Outcome Detect compatible/incompatible drift, normalize timestamps to UTC, quarantine invalid rows and avoid silently coercing bad values.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp 2026 — From APIs to Warehouses with dlt
Assigned scope: Long-form main class — schema normalization/change validation; book adds compatibility/time-zone/quarantine rules

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

Note: Use current workshop plus our stricter data-contract exercises.

[Open source](#)

7. Follow with me once

Process drift_events.csv. Parse amount + event_time; normalize valid times to UTC. Quarantine bad numeric and bad timestamp rows. Reconcile all six source rows. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

11. If you get stuck

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

12. Small independent task

Write a contract validator that accepts extra optional field `source_note`, but quarantines invalid amount/time. Normalize valid timestamps to UTC and preserve raw row.

13. Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

14. Teach it back

Explain schema drift, compatibility, time zones and quarantine in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success. Current authoritative reference(s) Python 3.14 `datetime/timezone` Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

M08 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-13

DE08-13 - Checkpoint/state management and replay safety

1. Why it matters

This matters because later Files, APIs & Data Ingestion work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Persist checkpoint state only after successful durable output and prove a crash/replay cannot skip uncommitted data. Teach it from zero Checkpoint state is part of correctness. If you advance a cursor before output is durably committed, a crash can make the next run skip data forever. Correct order is: read from old state -> write/validate -> commit output -> atomically persist new state. Replay from an old state must be safe.

3. Prerequisite / starting state

DE08-12 mastered.

4. Picture it in your head

VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt Stateful/incremental concepts; dlt state docs are current reference. Starting state / prerequisite DE08-12 mastered.

5. New words before we use them

VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt Stateful/incremental concepts; dlt state docs are current reference. Starting state / prerequisite DE08-12 mastered. Outcome Persist checkpoint state only after successful durable output and prove a crash/replay cannot skip uncommitted data.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp 2026 — From APIs to Warehouses with dlt

Assigned scope: Long-form main class — incremental/stateful ingestion; course task proves checkpoint/replay safety

How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/follow when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.

Note: No handwritten notes: save runnable state/checkpoint evidence instead.

[Open source](#)

7. Follow with me once

Simulate destination failure after candidate I006 is read but before commit. Prove stored state remains I005; rerun should read I006 again rather than skip it. Open the matching Guided Labs section.

Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

11. If you get stuck

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

12. Small independent task

Write pseudocode or Python showing state is saved only after destination success. Inject a failure before commit and prove state did not advance.

13. Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

14. Teach it back

Explain checkpoint/state management and replay safety in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success. Current authoritative reference(s) dlt advanced state management Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

M08 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-14

DE08-14 - Idempotent landing and content identity

1. Why it matters

This matters because later Files, APIs & Data Ingestion work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Use stable business/content identity so rerunning the same input does not create duplicate output while changed content remains detectable. Teach it from zero Idempotency means repeating the same operation produces the same final state. For files, SHA-256 can be a strong content identity signal; business/event keys may be the dedup identity for records. Do not confuse filename with content identity: a renamed duplicate file can still be identical bytes.

3. Prerequisite / starting state

DE08-13 mastered.

4. Picture it in your head

VIDEO FIRST route PyCon US 2025 - Building Resilient Data Pipelines: The Power of Idempotency Whole focused talk; then implement course content-identity/idempotent landing task. Starting state / prerequisite DE08-13 mastered.

5. New words before we use them

VIDEO FIRST route PyCon US 2025 - Building Resilient Data Pipelines: The Power of Idempotency Whole focused talk; then implement course content-identity/idempotent landing task. Starting state / prerequisite DE08-13 mastered. Outcome Use stable business/content identity so rerunning the same input does not create duplicate output while changed content remains detectable.

6. Useful video / source

Source: PyCon US 2025 — Building Resilient Data Pipelines: The Power of Idempotency

Assigned scope: Full ~25-minute talk — pitfalls → idempotent design → pytest strategy

How to use it: Watch the talk, then implement the course idempotent landing/content-identity task. Watching alone does not pass the lesson.

Note: PyCon US 2025 program confirms topic and presenter.

[Open source](#)

7. Follow with me once

Hash all landed batch files, then run landing twice. Prove destination contains one copy per intended file/content and hashes stay unchanged. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

11. If you get stuck

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

12. Small independent task

Use content hash + event key to prevent duplicates. Test same filename/same bytes, renamed/same bytes and same filename/changed bytes; define each outcome.

13. Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

14. Teach it back

Explain idempotent landing and content identity in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success. Current authoritative reference(s) Python 3.14 hashlib Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

M08 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-15

DE08-15 - Late-arriving data, event time and processing time

1. Why it matters

This matters because later Files, APIs & Data Ingestion work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Distinguish event time from arrival/processing time, classify lateness against an explicit tolerance and explain watermark trade-offs. Teach it from zero Event time says when the source event happened; processing/arrival time says when the pipeline saw it. Late data arrives after the allowed delay. Watermarks are progress assumptions used by streaming systems; a longer allowance improves completeness but increases latency. M08 teaches the concept without requiring Flink runtime.

3. Prerequisite / starting state

DE08-14 mastered.

4. Picture it in your head

VIDEO FIRST route Confluent Developer - Event Time and Watermarks 12-minute lesson: event vs processing time, out-of-order events, watermarks and late data. Starting state / prerequisite DE08-14 mastered.

5. New words before we use them

VIDEO FIRST route Confluent Developer - Event Time and Watermarks 12-minute lesson: event vs processing time, out-of-order events, watermarks and late data. Starting state / prerequisite DE08-14 mastered. Outcome Distinguish event time from arrival/processing time, classify lateness against an explicit tolerance and explain watermark trade-offs.

6. Useful video / source

Source: Confluent Developer — Event Time and Watermarks

Assigned scope: 12-minute video: event time vs processing time, out-of-order events, watermarks and late data

How to use it: Use the official video to see why event time differs from processing time; course task applies the concept without requiring Flink mastery.

Note: Official Confluent course page with hands-on exercises; current as audited 2026-08-28.

[Open source](#)

7. Follow with me once

For `late_events.json`, classify each event using 10-minute allowed lateness. Save IDs and delay seconds of late events. Explain why sorting only by `arrived_at` changes the business timeline. Open the matching Guided Labs section. Reproduce the pattern once with the normal `ParcelFlow` fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

11. If you get stuck

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

12. Small independent task

Classify the six late-event fixtures at 10-minute tolerance. Recompute at 20 minutes and explain completeness-vs-latency trade-off.

13. Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

14. Teach it back

Explain late-arriving data, event time and processing time in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success. Current authoritative reference(s) Confluent Event-Time Processing Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

M08 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-16

DE08-16 - Ingestion observability: counts, latency and run metadata

1. Why it matters

This matters because later Files, APIs & Data Ingestion work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Emit run-level evidence for rows seen/accepted/quarantined, retries, bytes, latency, checkpoint and status so failures can be diagnosed. Teach it from zero Observable ingestion emits evidence, not just "success". At minimum capture run_id/time, source, input rows/bytes, accepted/quarantine counts, retry count, elapsed/latency, checkpoint before/after, destination result and final status/error. Metrics must reconcile and should not leak credentials.

3. Prerequisite / starting state

DE08-15 mastered.

4. Picture it in your head

metadata VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt Validation/dashboard/run metadata concepts; course lab emits exact observability fields. Starting state / prerequisite DE08-15 mastered.

5. New words before we use them

metadata VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt Validation/dashboard/run metadata concepts; course lab emits exact observability fields. Starting state / prerequisite DE08-15 mastered. Outcome Emit run-level evidence for rows seen/accepted/quarantined, retries, bytes, latency, checkpoint and status so failures can be diagnosed.

6. Useful video / source

Source: DataTalksClub DE Zoomcamp 2026 — From APIs to Warehouses with dlt
Assigned scope: Long-form main class — pipeline metadata/validation/dashboard concepts; course lab adds counts/latency/run metadata
How to use it: Long or short video is allowed when it teaches the competency well. Watch actively without handwritten notes; type/run/look when relevant. Use the Teaching Book as ready-made notes. Close video/review/AI solving before the independent task.
Note: Evidence should be output/logs, not copied lecture notes.
[Open source](#)

7. Follow with me once

Create one run record from the normal smoke job. Verify seen = accepted + quarantine where applicable and retries are visible rather than hidden. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParquetFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

11. If you get stuck

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

12. Small independent task

Emit a JSONL run record with counts, bytes, retries, elapsed_ms, cursor_before/after, status and quarantine path. Prove no token/password is logged.

13. Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

14. Teach it back

Explain ingestion observability: counts, latency and run metadata in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success. Current authoritative reference(s) Python 3.14 logging Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

M08 Teaching Book

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE08-17

DE08-17 - Ingestion integration, reconciliation and backfill proof

1. Why it matters

This matters because later Files, APIs & Data Ingestion work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Run an end-to-end ingestion job, rerun it idempotently, backfill historical data and reconcile destination records to source/control totals. Teach it from zero Integration proves the pieces work together: inventory inputs -> land raw -> parse/validate -> extract/fetch -> deduplicate -> commit -> checkpoint -> observe -> reconcile. Then rerun unchanged input and perform a historical backfill. A backfill is not allowed to corrupt current state or duplicate already committed business events.

3. Prerequisite / starting state

DE08-01 through DE08-16 mastered.

4. Picture it in your head

proof VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt Whole-workflow review only; integrated build remains independent and local. Starting state / prerequisite DE08-01 through DE08-16 mastered.

5. New words before we use them

proof VIDEO FIRST route DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt Whole-workflow review only; integrated build remains independent and local. Starting state / prerequisite DE08-01 through DE08-16 mastered. Outcome Run an end-to-end ingestion job, rerun it idempotently, backfill historical data and reconcile destination records to source/control totals.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use matching data-ingestion / script / database chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Run `m08_smoke_runner.py` from a clean normal/output folder. Save the generated JSON evidence. Rerun it and confirm landing/idempotency controls remain stable. Then explain how you would add one older batch without resetting unrelated state. Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

8. Expected result / what you should see

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

11. If you get stuck

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

12. Small independent task

From a clean output folder run the full normal ingestion smoke, rerun unchanged inputs, then add one hypothetical older batch. Write a reconciliation table and backfill safety explanation.

13. Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

14. Teach it back

Explain ingestion integration, reconciliation and backfill proof in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success. Current authoritative reference(s) dlt REST API tutorial dlt incremental loading Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: Proceed to the module practical/Gate route after the module mastery checklist.

Module mastery checklist + revision quick reference

- ☐ Every lesson has a saved genuine independent attempt.
- ☐ Every lesson has at least one independent validation/control appropriate to the tool.
- ☐ Protected reviews were opened only after attempts; any weak lesson has a changed retry after review was closed.
- ☐ You can teach back each lesson in your own words and name one realistic failure mode.
- ☐ Required module practical/retest evidence is saved and no critical defect is unresolved.
- ☐ D+1, D+4, D+10 and D+30 revision events are scheduled for weak or high-value competencies.

Revision cadence

D+1: fresh micro-task. D+4: mixed retrieval. D+10: changed/failure case. D+30: interview/project retrieval. Always attempt from memory before reopening a source.

Release boundary

This book is a static learner artifact. It does not prove live tool/cloud execution or a protected Gate pass. Runtime evidence must be recorded separately when the curriculum requires it.